

Git mínimo para este TP

Una sola vez

1. Crear un repo vacío en GitHub (nombre sugerido: **ayed-2026-pokedex**, **ayed-2026-recetario** o **ayed-2026-musica**). No uses el repo de la consigna de la cátedra.
2. Copiar esta carpeta **esqueleto** (no el de los otros grupos).
3. Subir:

```
git init
git add .
git commit -m "E1: esqueleto y datos del tema"
git branch -M main
git remote add origin https://github.com/USUARIO/ayed-2026-TEMA.git
git push -u origin main
```

Invitar a los integrantes. El grupo se avisa **sí o sí** por mail a:

- diego.ambrossio@unab.edu.ar
- angel.bianco@unab.edu.ar

Asunto y cuerpo: ver "**Correo de cada entrega**" más abajo (mail inicial del grupo). Incluir nombres, mails, usuarios de GitHub, tema y URL del repo.

Preentrega (opcional, domingo 30-ago)

Solo para probar que el repo se puede clonar. No suma nota.

```
git tag preentrega
git push origin preentrega
```

Después mandan el mail con la URL. El código del catálogo no hace falta todavía.

Cada entrega

```
git add .
git status
git commit -m "E3: lista enlazada, pila y cola"
git tag entrega-3
git push
git push origin entrega-3
```

El mensaje de commit puede ser el que quieran; el **tag** tiene que ser exactamente **entrega-1 ... entrega-6**.

Si se equivocan antes del vencimiento:

```
git tag -d entrega-3
git push origin :refs/tags/entrega-3
git tag entrega-3
git push origin entrega-3
```

Después del domingo 23:59 no retaguear en silencio: ese tag ya se está corrigiendo.

Correo de cada entrega

Avisá por mail a Ambrossio y Bianco **con este formato**, tageado y pusheado.

Mail inicial del grupo (integrantes, tema y repo):

```
Asunto: AyED C2 2026 Comision X - GRUPO XX - Armado
Repo: https://github.com/usuario/ayed-2026-tema
Integrantes:
- Nombre Apellido - mail@unab.edu.ar - @usuario_github
- Nombre Apellido - mail@unab.edu.ar - @usuario_github
Tema: Pokédex / Recetario / Biblioteca musical
```

Asunto de cada entrega: **AyED C2 2026 Comision X - GRUPO XX - Entrega X**

Cuerpo:

```
Repo: https://github.com/<usuario>/<repo>
Entrega de <etapa>: <especificación corta de lo que incluye el tag>
```

Ejemplos por etapa:

Entrega	Especificación corta para el cuerpo
E1	esqueleto clonado y funcional, tema en config.py , catálogo armado a mano (los .txt se usan desde E5), CLI que lista el catálogo, README con integrantes e informe arrancado
E2	código en módulos, clases del dominio, recursión del tema con traza en el informe y protocolo de pruebas con 8 casos escritos
E3	ListaEnlazada propia con iterador, Pila y Cola sobre esa lista, colección principal del dominio con tope, excepciones propias y protocolo ejecutado

Entrega	Especificación corta para el cuerpo
E4	búsqueda lineal y binaria propias, dos ordenamientos propios, tabla de complejidad y mediciones con <code>time.perf_counter</code>
E5	persistencia de texto simple (<code>.txt</code>) y binario con <code>struct</code> (registros de longitud fija), actualización por posición y alta/baja/modificación persistente
E6	producto cerrado con las 10 operaciones del menú, informe completo, protocolo de regresión (15 casos) y declaración de IA final

El tag que se corrige es el de la etapa (`entrega-1 ... entrega-6`), no un commit suelto de `main`.

Qué no subir

`__pycache__`/, `.venv`/, archivos `.bin` de prueba locales (el programa los tiene que **poder generar**). El `.gitignore` del esqueleto ya cubre lo habitual.